
Unifor

Paintball - Kattis

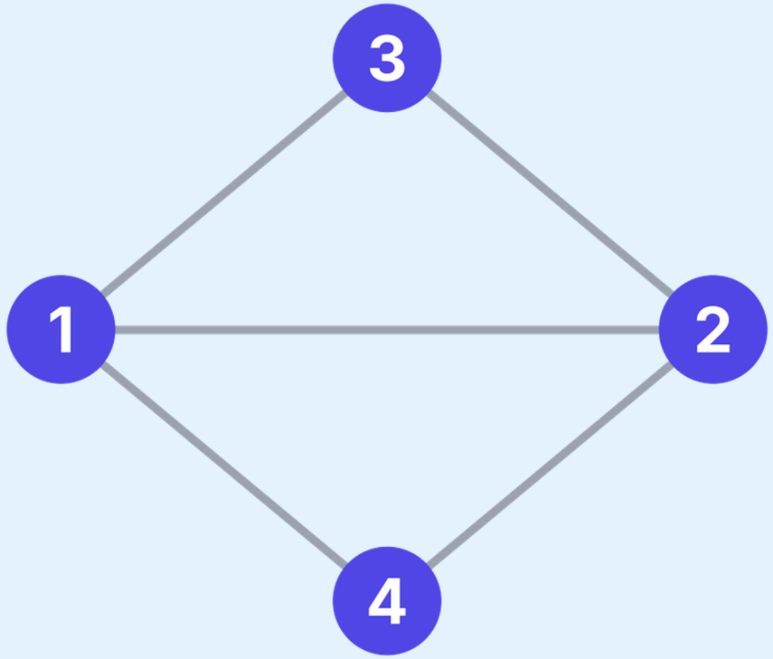
Emparelhamento em grafo bipartido e algoritmo de Hopcroft-Karp

Alunos: Karlos Eduardo Sousa Pinto - 2320262
Maria Eduarda Coutinho Angelim - 2324319
Orientador: Prof. Me. Ricardo Carubbi
Disciplina: Resolução prob. com grafos
Centro de Ciências Tecnológicas - CCT
Universidade de Fortaleza - UNIFOR

O Problema

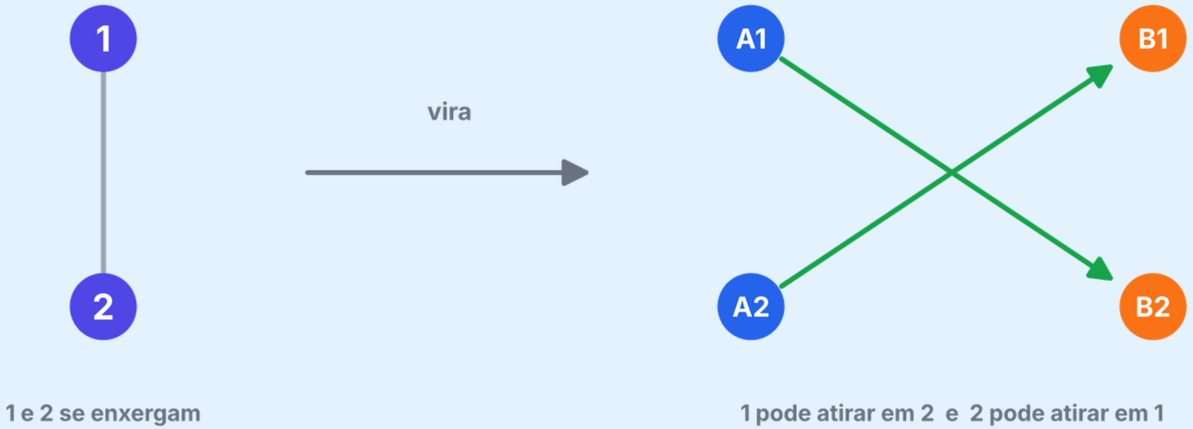
Paintball: todos atingidos, uma vez só

- N jogadores e M pares que se enxergam (visão mútua).
- Cada um tem 1 bala e só atira em quem vê.
- Todos tem que ser atingidos uma vez, senão **IMPOSSIBLE**.



E:	**S:**
4 5	4 <- alvo do jogador 1
1 2	3 <- alvo do jogador 2
1 3	1 <- alvo do jogador 3
1 4	2 <- alvo do jogador 4
2 3	
2 4	

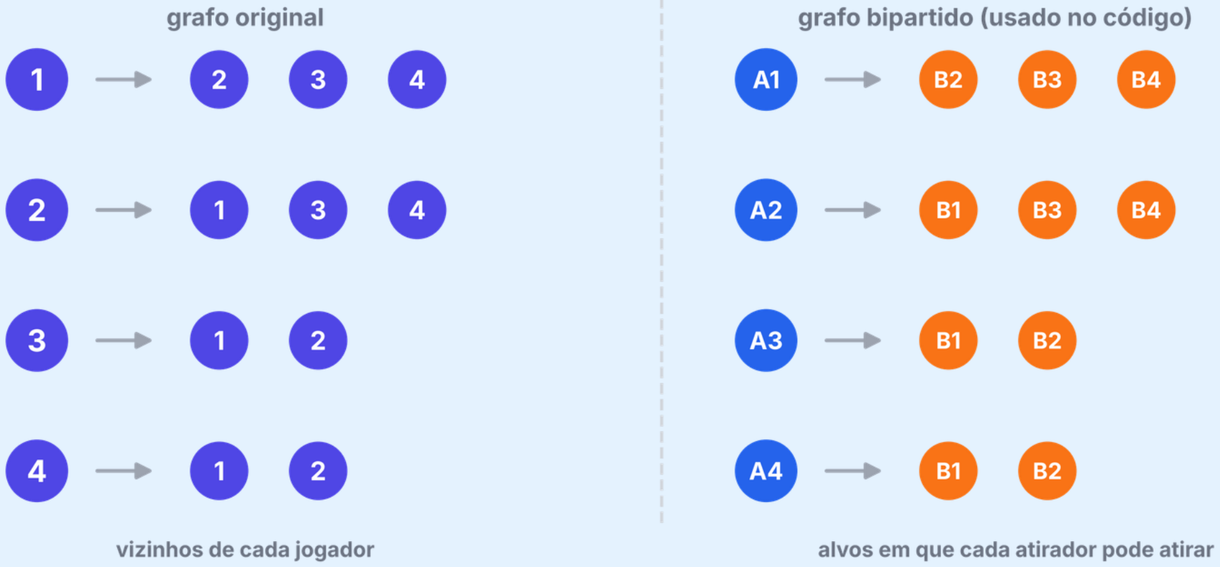
Modelagem: Grafo Bipartido



1 e 2 se enxergam

1 pode atirar em 2 e 2 pode atirar em 1

```python
def constroi_grafo_bipartido(n, pares):
    G = Graph(2 * n)
    for a, b in pares:
        if a != b:  G.add_edge(a, n + b)
        if b != a:  G.add_edge(b, n + a)
    return G
```



vizinhos de cada jogador

alvos em que cada atirador pode atirar

- Existe resposta ⇔ existe emparelhamento perfeito

Hopcroft-Karp

- Caminho aumentante: Inverter o caminho dá +1 no emparelhamento.
- Fluxo: toda aresta tem capacidade 1 e a emparelhada está saturada.
- Cada fase tem uma **BFS** que mede os níveis e uma **DFS** que colhe caminhos mínimos disjuntos.

Resíduo

```
1       def _is_residual_graph_edge(self, v, w):
2           if self.mate[v] != w and self.bipartition.color(v):
3               return True
4           if self.mate[v] == w and not self.bipartition.color(v):
5               return True
6           return False
```

Grafo de níveis

```
1       def _is_level_graph_edge(self, v, w):
2           return self.dist_to[w] == self.dist_to[v] + 1 and self._is_residual_graph_edge(v, w)
```

- **Custo:O(E√V)**

BFS e DFS

- BFS: mede os níveis. Não acha caminho.
- DFS: usa os níveis e colhe vários caminhos disjuntos.

BFS

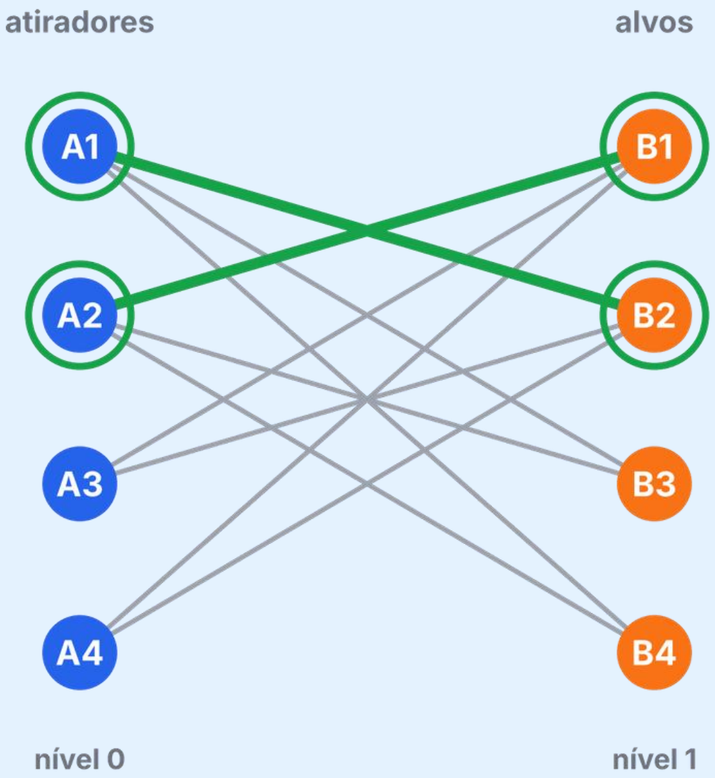
```
1
2   for v in range(V):
3       if color(v) and not matched(v):
4           fila.append(v); dist_to[v] = 0
5
6   while fila:
7       v = fila.popleft()
8       for w in adj(v):
9           if residual(v, w) and not marked[w]:
10              dist_to[w] = dist_to[v] + 1
11              if not matched(w): achou = True
12              if not achou: fila.append(w)
```

DFS

```
1   path = [s]
2   while path:
3       v = path[-1]
4       if proximo[v] >= len(adj(v)):
5           path.pop(); continue
6       w = adj(v)[proximo[v]]; proximo[v] += 1
7       if not level_edge(v, w): continue
8       path.append(w)
9       if not matched(w):
10          while path:
11              x = path.pop(); y = path.pop()
12              mate[x] = y; mate[y] = x
```

Execução

- **Fase 1**



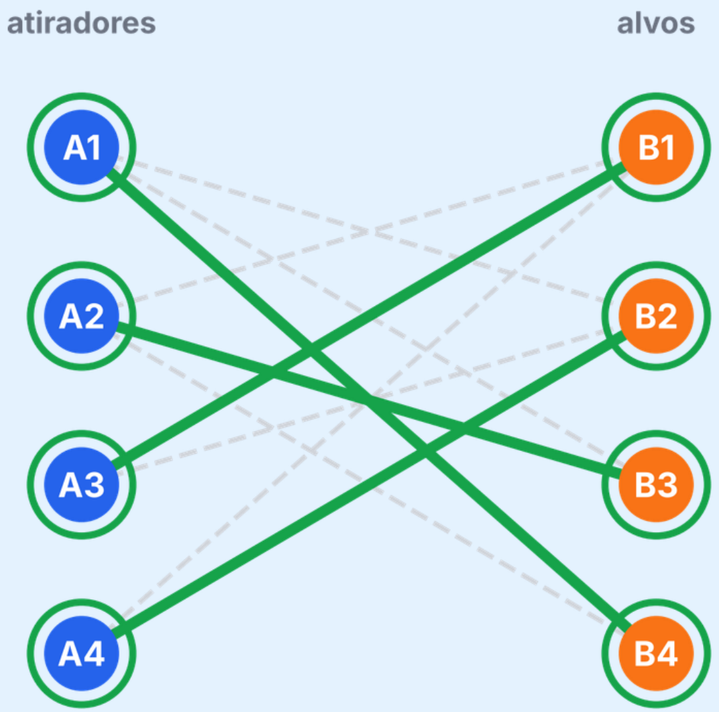
- **Fase 2.1: Grafo de Níveis**

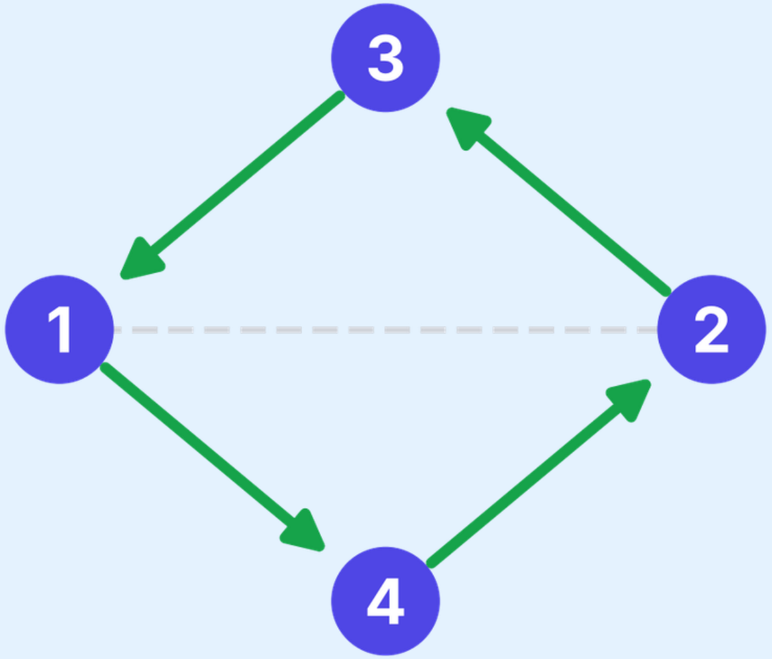


- **Fase 2.2: Caminhos Aumentantes**



Resolução






Unifor